TCN Software Validation Report

Author	Ryan Gaffere
Version	v1.0
Date	May 5th, 2026

Validation of predictive performance and numerical equivalence between the C++ inference engine and a PyTorch reference implementation.

Overview

This report validates a Temporal Convolutional Network (TCN) for streaming prediction on 6-channel IMU data (accelerometer and gyroscope). The model is causal and predicts the next timestep (t -> t+1) using only current and past inputs.

Two aspects are evaluated:

1. The model's ability to produce meaningful predictions on time-series data
2. The correctness of a custom C++ inference engine

The C++ implementation is compared against a reference model in PyTorch using identical weights and input data to verify numerical equivalence.

The goal is to confirm both predictive functionality and implementation accuracy, establishing the C++ engine as a reliable software baseline for future hardware development.

Architecture

Parameter	**Value**
Model type	Temporal Convolutional Network
Input channels	6
Output channels	6
Residual blocks	7
Hidden channels	16
Kernel size	3
Dilations	1, 2, 4, 8, 16, 32, 64
Convolutions per block	2
Activation	ReLU
Prediction target	Next timestep, t -> t+1

Receptive field	509 samples
Inference mode	Streaming, latest timestep only

The model uses a causal TCN architecture, meaning each prediction only depends on the current and previous input samples. This prevents future data leakage and makes the network suitable for real-time sensor prediction.

Each residual block contains two dilated causal convolutions with the same dilation factor. The dilation increases by powers of two across blocks, allowing the model to capture long-term temporal behavior while keeping the kernel size small.

Residual connections are used to pass information across blocks and stabilize training. The final output layer maps the hidden representation back to 6 channels so the model can predict the next value of each IMU signal.

One residual block follows this form:

```
Convolution 1 -> ReLU -> Convolution 2 -> ReLU -> Residual Add
```

Note that the Residual Add is added after the second ReLU. The model follows the Temporal Convolutional Network architecture described in *Bai et al. (2018).*

Please refer to *"TCN System Overview"* for the reasoning behind this network architecture and its parameters.

Validation

The software validation was split into two checks: first, confirming that the trained TCN can produce meaningful next-timestep predictions, and second, confirming that the custom C++ inference engine matches the PyTorch reference implementation.

Prediction Validation

The trained PyTorch model was evaluated on held-out validation data to verify that the network learned the time-series behavior of the input signals. Model performance was measured by comparing predicted next-timestep values against the true next samples.

This step confirms that the architecture and trained weights produce useful predictions before validating the standalone C++ implementation.

Dataset

The validation dataset contains 6-channel IMU sequences, consisting of 3-axis accelerometer data and 3-axis gyroscope data. Each input window is passed into the model, and the target output is the next timestep for all 6 channels.

The same validation data was used for both the PyTorch reference model and the C++ inference engine.

C++ Equivalence Validation

After prediction validation, the trained weights were exported from PyTorch and loaded into the custom C++ inference engine. The same input samples were then passed through both implementations.

The outputs were compared sample-by-sample and channel-by-channel using numerical error metrics:

Metric	Purpose
Max absolute error	Captures the largest single output difference
Mean absolute error	Measures average output deviation
RMSE	Measures overall numerical mismatch

These metrics are provided in the next section titled “Results.”

Results

Prediction Performance

The model was trained for up to 50 epochs with early stopping applied based on validation loss. Training converged at epoch 27 with a best validation MSE of 0.006528.

This corresponds to an approximate RMSE of:

$$\sqrt{0.006528} = 0.0808$$

indicating an average prediction error of ~ 0.08 per channel.

Validation was performed on $61{,}261$ samples across 6 channels, confirming the model's ability to generalize to unseen data.

Qualitative evaluation further supports this result. As shown in Figures 1–2, the predicted signals closely track the ground truth across time, capturing both low-frequency trends and high-frequency variations. The model is able to follow rapid changes in the signal, with larger deviations primarily occurring at sharp transitions or spikes.

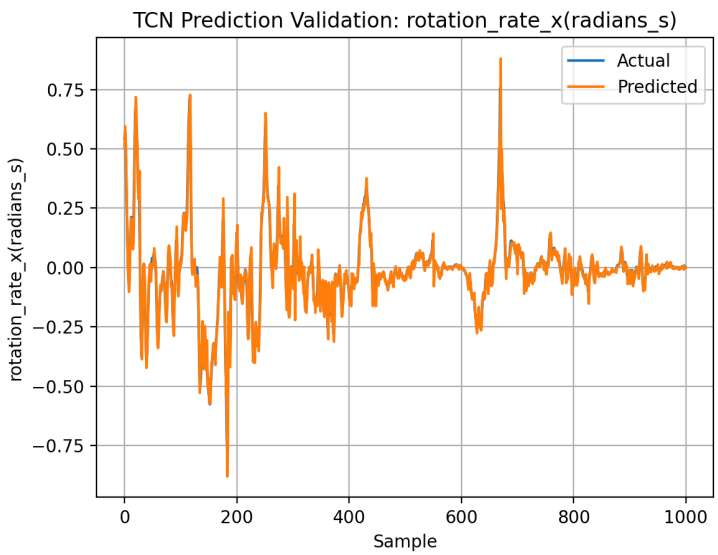

Figure 1: Gyroscope X, Predicted vs Actual

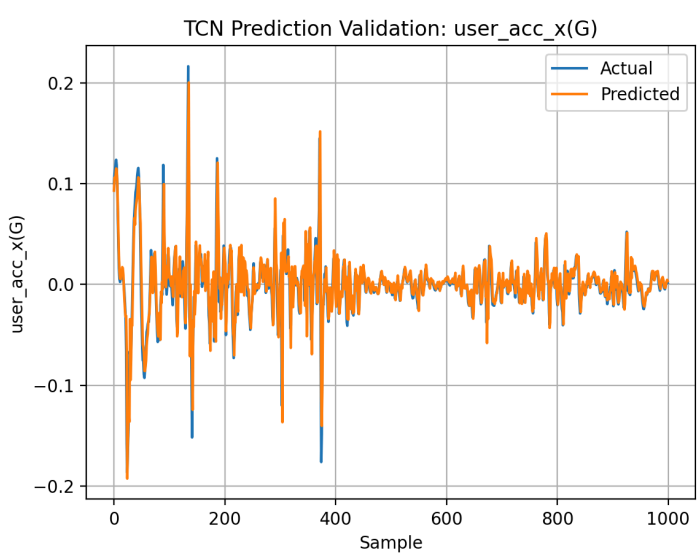

Figure 2: Accelerometer X, Predicted vs Actual

Prediction error over time (Figure 3) shows that most errors remain low, with occasional spikes corresponding to sudden signal changes. This behavior is expected for next-step prediction in noisy time-series data and indicates that the model has learned the underlying temporal structure rather than simply smoothing the signal.

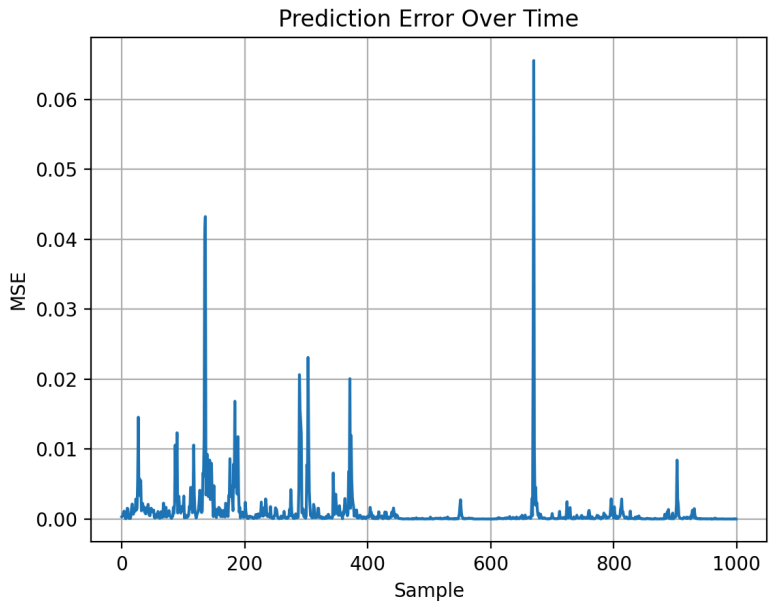

Figure 3: Prediction error over time

C++ Equivalence Validation

The trained model weights (~415 KB) were exported from PyTorch and loaded into the custom C++ inference engine. Both implementations were evaluated on identical input data.

The numerical difference between outputs is:

Metric	Value
Max Absolute Error	2.3727×10^{-6}
Mean Absolute Error	1.245×10^{-7}
RMSE	1.769×10^{-7}

These errors are within floating-point precision limits, confirming that the C++ implementation is numerically equivalent to the reference model.

The validation passed all equivalence checks.

Warmup Behavior

A discrepancy is observed during the initial ~509 samples, corresponding to the model's receptive field.

- The PyTorch implementation uses zero-padding for missing history
- The C++ implementation skips out-of-range inputs

Once the internal buffer is fully populated, both implementations produce consistent outputs.

Implementation Notes

Streaming inference

- Latest-timestep prediction only (t -> t+1)
- State maintained across timesteps
- Ring buffer used for O(1) updates (no shifting)

Dilated convolution

- Implemented via indexed access into buffer
- Supports arbitrary dilation without expanding tensors
- Out-of-range taps skipped (no zero-padding in C++)

Residual blocks

- Two dilated convolutions per block
- ReLU applied after each convolution
- Residual connection added at block output

Numerical precision

- Double-precision floating point used
- Matches PyTorch reference behavior

Deterministic, dependency-free

- Fixed architecture, configurable weights
- Designed for portability to hardware implementation

Conclusion

The TCN model demonstrates effective predictive performance on multi-channel IMU data, accurately capturing the temporal behavior of the input signals. Validation results confirm that the model produces meaningful next-timestep predictions across all channels.

The custom C++ inference engine has been validated against a PyTorch reference model, with errors on the order of floating-point precision, confirming correct implementation. This provides a software golden model for hardware verification.

With both predictive capability and numerical equivalence verified, the C++ inference engine serves as the primary reference model and is suitable to benchmark accuracy and behavior in the hardware implementation phase.